Technical Report

Malware Development and Remote Access

Student Name: Mahrukh
Environment: Controlled & Isolated Lab Setup

Abstract

This report presents an educational malware development lab conducted in a controlled and isolated environment. The purpose of the experiment was to understand how reverse shell payloads function on Linux and Android systems and how such activities can be monitored and analyzed from a defensive perspective. Kali Linux was used as the attacker machine, Ubuntu Linux and an Android device were used as victim systems, and Python scripts were developed to establish reverse shell connections. Wireshark was utilized to analyze the network traffic generated during the attack simulations. The lab was performed strictly for academic learning and cyber defense understanding.

Lab Environment Setup

The lab environment was designed to ensure isolation and safety. VirtualBox was used as the virtualization platform to host the operating systems.

2.1 Kali Linux (Attacker Machine)

- Role: Listener and command execution system
- Network Mode:
  - Host-Only Adapter (for Ubuntu testing)
  - Bridged Adapter (for Android testing)
- Tools Used:
  - Netcat (listener)
  - Wireshark (traffic analysis)
  - Python (scripting)

2.2 Ubuntu Linux (Victim Machine)

- Role: Linux victim system
- Network Mode: Host-Only Adapter
- Purpose: Execute Python reverse shell script and establish a session with Kali Linux

2.3 Android Device (Victim Device)

- Device: Samsung Smartphone
- Android Version: Android 15
- Network Mode: Same network as Kali (via Bridged Adapter)
- Purpose: Execute Python-based reverse shell logic (via supported environment) and connect back to Kali Linux

---

Network Configuration

Two different network configurations were used:

- **Host-Only Network:**
  Used between Kali Linux and Ubuntu Linux to ensure complete isolation from the external internet.
- **Bridged Network:**
  Used between Kali Linux and the Android device so both devices could communicate over the same local network.

This dual configuration allowed controlled testing while maintaining flexibility for Android connectivity.

Design Architecture

The lab architecture consisted of three main components: the attacker system, victim systems, and the monitoring tool. Kali Linux acted as the attacker and listener machine. Ubuntu Linux and a Samsung Android device (Android version 15) acted as victim systems. For Ubuntu testing, a Host-Only network was used to ensure complete isolation. For Android testing, Kali Linux was configured with a Bridged network so it could communicate with the mobile device on the same local network. Python-based reverse shell scripts were deployed on both victim systems, while Wireshark was used on Kali Linux to capture and analyze network traffic during the sessions.

Code Explanation

The reverse shell programs were written in Python using socket programming. The script initiates a TCP connection from the victim system to the Kali Linux machine on a predefined IP address and port. Once the connection is established, the script listens for commands sent by the attacker, executes them on the victim system, and sends the output back to Kali Linux. On the Ubuntu system, executed commands were logged locally to support forensic analysis. On the

Android device, the script was designed to maintain connectivity and basic logging while running in the background. The code was kept simple and non-destructive to comply with ethical and academic guidelines.

Attack Flow (Attacker → Victim → Session)

The attack flow begins with the attacker configuring a listener on Kali Linux. The victim system then executes the Python reverse shell script. The victim initiates an outbound connection to the attacker, bypassing inbound firewall restrictions. Once the connection is accepted, a remote shell session is established. The attacker can then execute commands on the victim system and receive command output in real time. All communication between the attacker and victim is transmitted over the network and can be observed using Wireshark.

Defense Strategies

Understanding reverse shell behavior helps in designing effective defense mechanisms. Defensive strategies include monitoring outbound network connections, deploying intrusion detection and prevention systems, using endpoint security solutions, enforcing application whitelisting, and analyzing logs for suspicious activity. Network traffic analysis tools such as Wireshark can help security teams identify unusual communication patterns that may indicate malicious behavior.

Ethical Considerations

This experiment was conducted strictly for educational purposes within a controlled lab environment. All systems used were owned by the student, and no external networks or real users were involved. The reverse shell scripts were non-destructive and designed only to demonstrate concepts. The lab emphasizes responsible use of cyber security knowledge for defense, detection, and awareness rather than real-world exploitation.

Conclusion

The lab successfully demonstrated the working of reverse shell payloads on Linux and Android platforms using Python. By configuring isolated networks, establishing remote sessions, and analyzing traffic with Wireshark, the experiment provided practical insight into attacker techniques and defensive analysis. This hands-on experience strengthens understanding of

malware behavior and highlights the importance of ethical responsibility in cyber security practices.
